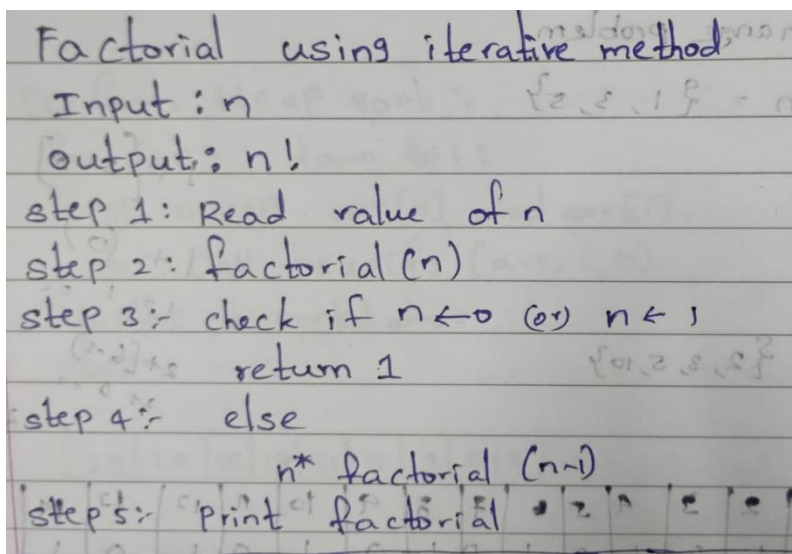


PRACTICAL - 4

AIM : Implementation and Time analysis of factorial program using iterative and recursive method

FACTORIAL USING ITERATIVE METHOD

ALGORITHM:



Factorial using iterative method
Input : n
Output : n!
step 1: Read value of n
step 2: Factorial(n)
step 3: check if $n \leq 0$ (or) $n \leq 1$
return 1
step 4: else
n * factorial(n-1)
step 5: print factorial

CODE:

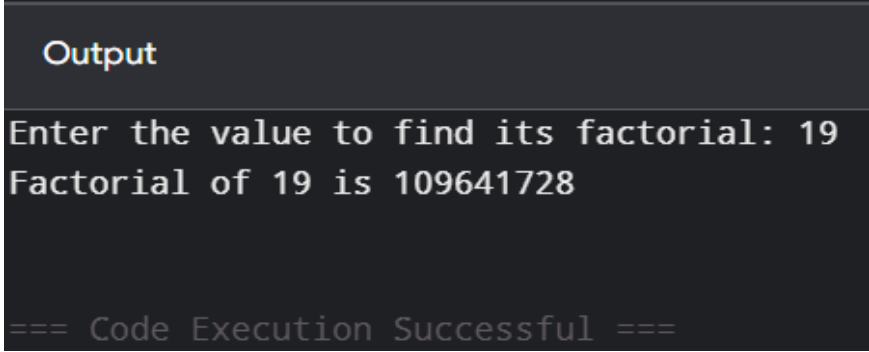
```
#include <iostream>

using namespace std;

class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;
        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }
    }
}
```

```
}  
    return result;  
    }  
};  
int main()  
{  
    int n,result;  
    Factorial obj;  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
    result = obj.fact(n);  
    cout << "Factorial of " << n << " is " << result << endl;  
    return 0;  
}
```

OUTPUT:



The screenshot shows a terminal window with a dark background. At the top, the word "Output" is written in a light blue font. Below it, the program's output is displayed in a white monospaced font: "Enter the value to find its factorial: 19" followed by "Factorial of 19 is 109641728". At the bottom of the terminal, the text "=== Code Execution Successful ===" is shown in a light gray font.



TIME COMPLEXITY:

Best case:

```
int factorial(int n) {  
    int fact = 1;  
    for (int i = 1; i <= n; i++) {  
        fact = fact * i;  
    }  
    return fact;  
}
```

$T(n) = 1 + 1 + (n+1) + n + n+1$
 $= 3n + 4$
 $T(n) = O(n)$

Average case:

```
int factorial(int n) {  
    int fact = 1;  
    for (int i = 1; i <= n; i++) {  
        fact = fact * i;  
    }  
    return fact;  
}
```

$T(n) = 1 + 1 + (n+1) + n + n+1$
 $= 3n + 4$
 $T(n) = O(n)$



Worst case:

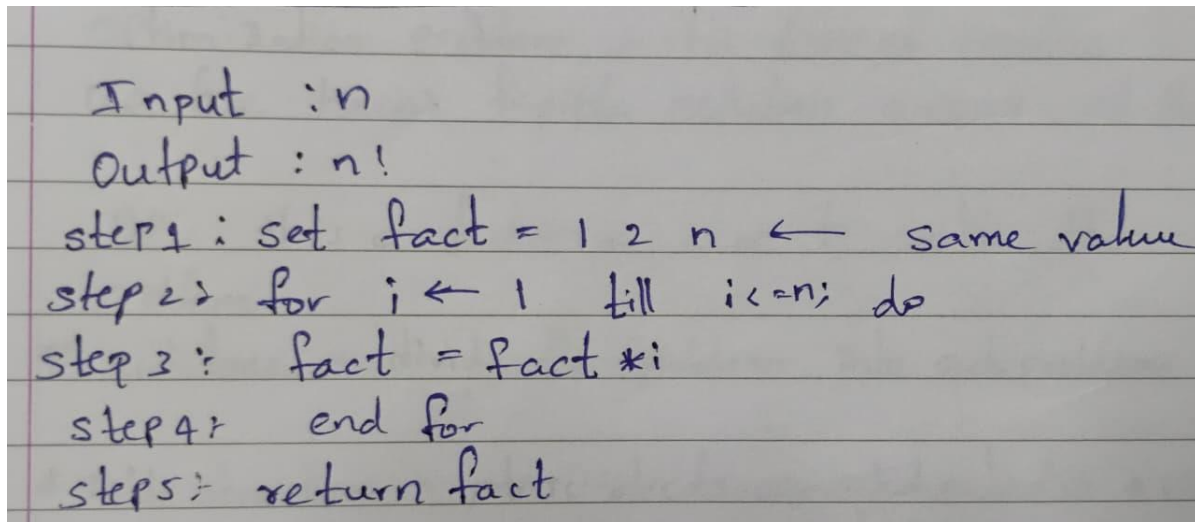
```
int factorial(int n) {  
    int fact = 1;  
    for (int i = 1; i <= n; i++) {  
        fact = fact * i;  
    }  
    return fact;  
}
```

$$T(n) = 1 + 1 + (n+1) + n + n+1$$
$$= 3n + 4$$
$$T(n) = O(n)$$



FACTORIAL USING RECURSIVE METHOD

ALGORITHM :



Input : n
Output : n!
step 1: set fact = 1 2 n ← same value
step 2: for i ← 1 till i ≤ n; do
step 3: fact = fact * i
step 4: end for
step 5: return fact

CODE:

```
#include <iostream>

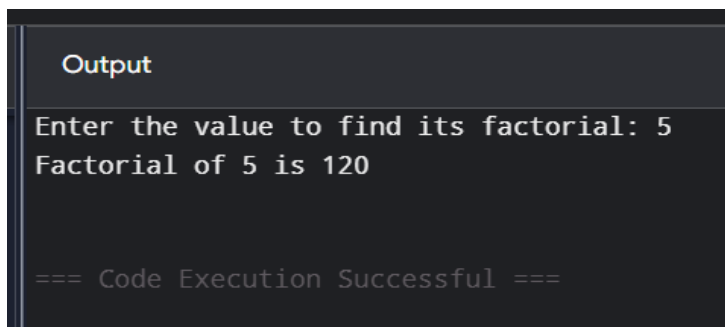
using namespace std;

class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else
```

```
{
    int result = n * factr(n - 1);
    return result;
}
};

int main()
{
    int n, result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.factr(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

OUTPUT:



TIME COMPLEXITY:

Best case :



Best case
if recursion calls only one time
int factorial (int n) {
if (n == 0 || n == 1) — 1
return 1; — 1
else
return n * factorial (n-1); — 1
}
 $T(n) = O(1)$

Average case :

Average case:-
int factorial (int n) { — 1
if (n == 0 || n == 1) — 1
return 1;
else
return n * factorial (n-1); — n
}
 $T(n) = n + 1 + 1 + 1$
 $T(n) = O(n)$



Worst case :

```
int factorial(int n) {  
    if (n == 0 || n == 1) return 1;  
    else  
        return n * factorial(n-1);  
}
```

$T(n) = n + 1 + 1 + 1$
 $T(n) = O(n)$